



Department of Artificial Intelligence and Machine Learning

Course Code: : AI254TA

Date

: 08.01.25

Semester : V

Time

: 2.00-4.00

Max Marks : 60

Duration

: 120 mins

Machine Learning Operations (MLOps)

CIE 2

Note: Answer all the Questions

SL. No	Questions	M	B T	CO
PART – A				
1	Give any two features of Clear ML	2	1	3
2	What is one hot encoding	2	1	1
3	“Transfer learning is a powerful technique in machine learning where a model developed for one task is reused as the starting point for a model on a second task”. Justify this statement with a real time application of transfer learning	2	2	2
4	Give any two needs for feature selection	2	1	1
5	What are two key factors to consider when managing interactions between models in MLOPs	2	2	2
PART – B				
1	A “Hyperparameters are critical elements in configuring machine learning algorithms”. Justify the statement with an example	5	2	5
	B Elaborate the different challenges faced by the ML algorithms in MLOPs.	5	2	2
2	A What the need for version management and reproducibility in developing a machine learning model. Give examples	6	2	2
	B Debate any two the challenges in choosing evaluation metrics when comparing different models for ML Model. Give examples to justify the answer.	4	2	2
3	a XYZ bank has a machine learning model predicting fraudulent transactions, but deployment and monitoring processes are manual and error-prone. This leads to: 1. Delays in releasing new model versions. 2. Lack of visibility into model performance post-deployment. 3. Limited automation in the pipeline. The goal is to implement an MLOps pipeline to automate the deployment process, monitor performance, and enable faster iterations. (i) List the various MLOPs tools that can be used to implement the pipe line.	10	3	3



Department of Artificial Intelligence and Machine Learning

		(ii) Identify and justify the tool you would prefer to deploy the model Elaborate in detail how do you implement MLOPs pipeline with reference quality assurance and testing during production.			
4	A	Paraphrase the statement “runtime environments can have a big effect on deployment of ML Deployment and Production” with reference to adaptation of runtime environment from development to production.	6	3	2
	B	Discuss the need of reproducibility and auditability in building a MLOPs pipeline	4	2	5
5	A	With a relevant example determine the attack and vulnerability that can affect an ML Model?	6	2	2
	B	Consider the scenario in Q3. If you were an MLOps engineer, how would you address a situation where a model behaves unexpectedly in production?	4	3	2

M-Marks, BT-Blooms Taxonomy Levels, CO-Course Outcomes

Marks Distribution	Particulars	C01	C02	C03	C04	C05	L1	L2	L3	L4	L5	L6
	Max Marks Part A	4	4	2			6	4				
	Max Marks Part B		27	14		9		34	16			

Course Outcomes: After completing the course, the students will be able to:-

C01	Identify and apply various ML-Ops techniques to deploy machine learning models for real-world problems.
C02	Design, deploy and evaluate Machine Learning models, follow the operational practices to benefit society, science, and industry.
C03	Use modern tools and techniques to organize ML model from development to production for real world problems
C04	Demonstrate effective communication through team presentations and reports to analyse the impact of the standard MLOPs practices on industry and society.
C05	Conduct performance evaluation, design, deploy models in par with the appropriate Governance for the benefit of the industry and society.

8.01.25 CIE 2: Quiz Solutions

PART - A (Short Answers)

1. Give any two features of ClearML

- **Experiment Tracking:** ClearML automatically logs and tracks code, hyperparameters, training metrics, console outputs, and artifacts, making every ML experiment fully reproducible.
- **MLOps Automation and Orchestration:** It allows seamless transition of code from a local machine to a remote cluster, automating pipelines, managing workloads, and scheduling tasks without requiring extensive infrastructure code.
- *(Additional feature)* **Data and Model Management:** It provides built-in version control for datasets and a centralized model registry to manage model versions and deployments.

2. What is one hot encoding?

- **Definition:** One hot encoding is a data preprocessing technique used in machine learning to convert categorical variables into a numerical format that ML algorithms can process.
- **How it works:** It creates a new binary column (0 or 1) for each unique category present in the original feature. For any given data point, the column corresponding to its category gets a value of 1 (hot), and all other newly created category columns get a value of 0 (cold).
- *Example:* A "Color" feature with values (Red, Blue, Green) becomes three columns: Color_Red , Color_Blue , and Color_Green .

3. "Transfer learning is a powerful technique in machine learning where a model developed for one task is reused as the starting point for a model on a second task". Justify this statement with a real-time application of transfer learning.

- **Justification:** Training deep neural networks from scratch requires massive amounts of labeled data, time, and computing power. Transfer learning is powerful because it allows a model to leverage the generalized features (like edge detection in images or grammar rules in text) it learned from a massive dataset, requiring only a small amount of specific data and time to fine-tune it for a new, related task.
- **Real-time application: Medical Image Analysis (e.g., Tumor Detection).** A Convolutional Neural Network (CNN) like ResNet, which was originally trained on millions of everyday images (ImageNet) to recognize general shapes and textures, is reused. Its base layers are frozen, and only the final layers are retrained on a much smaller dataset of MRI scans to accurately detect tumors.

4. Give any two needs for feature selection

1. **Reduces Overfitting:** By removing irrelevant, redundant, or noisy features, feature selection helps the model focus on the most important patterns, improving its ability to

generalize to new, unseen data.

2. **Improves Computational Efficiency:** Fewer features mean smaller datasets, which drastically reduces the time required for model training and inference, and lowers memory and storage requirements.
3. *(Additional need)* **Enhances Interpretability:** A model with fewer input variables is much easier for data scientists and business stakeholders to understand and explain.

5. What are two key factors to consider when managing interactions between models in MLOps?

1. **Data Dependencies and Cascading Failures:** When models are chained together (Model A's output becomes Model B's input), a slight degradation or drift in Model A can cause a severe, amplified failure in Model B. You must continuously monitor the intermediate data to catch cascading errors.
2. **Version Compatibility and API Contracts:** When updating a model in an interacting pipeline, the new version must maintain a strict API contract (input/output schema) to ensure it doesn't break downstream models that rely on a specific format or data type.

PART - B (Long Answers)

1(A) "Hyperparameters are critical elements in configuring machine learning algorithms". Justify the statement with an example. (5 Marks)

1. Introduction Hyperparameters are configuration variables that govern the entire learning process of a machine learning model. Unlike standard model parameters (such as weights and biases) which the algorithm learns directly from the training data, hyperparameters are set manually by the data scientist *before* training begins.

2. Justification of Criticality Hyperparameters are critical because they act as the "control knobs" of the algorithm, directly influencing:

- **Overfitting and Underfitting:** They dictate the complexity of the model. Poorly chosen hyperparameters will cause an algorithm to either memorize the noise in training data (overfit) or fail to capture the underlying patterns entirely (underfit).
- **Training Time and Resource Cost:** They determine how long an algorithm runs and how much computational power it requires to converge.
- **Algorithm Convergence:** In optimization algorithms (like Gradient Descent), specific hyperparameters ensure the model actually finds the optimal solution rather than oscillating infinitely or stopping prematurely.

3. Practical Example: Random Forest Algorithm Consider training a Random Forest algorithm to predict customer churn. Two highly critical hyperparameters are:

- `max_depth` **(Maximum depth of trees):** * If set too high (e.g., 100): The decision trees become too complex and deep. They learn the training data perfectly (including random outliers), resulting in severe **overfitting** and poor real-world predictions.

- *If set too low (e.g., 2):* The trees are too shallow to capture complex relationships in the customer data, resulting in **underfitting**.
- **n_estimators (Number of trees):** Setting this too low makes the model unstable, while setting it extremely high wastes compute resources with diminishing returns on prediction accuracy.

4. Conclusion Because an algorithm's ability to learn effectively and generalize to new data relies entirely on these configurations, hyperparameters are fundamentally critical. Tuning them systematically (e.g., via Grid Search or Random Search) is a mandatory step in achieving optimal model performance.

1(B) Elaborate the different challenges faced by the ML algorithms in MLOPS. (5 Marks)

1. Introduction While machine learning algorithms might perform exceptionally well in isolated, static development environments (like a local Jupyter Notebook), deploying and maintaining them in production via MLOps introduces severe, dynamic challenges that can degrade their performance and reliability.

2. Key Challenges Faced by ML Algorithms in MLOps

- **1. Data Drift and Concept Drift**
 - *Description:* Real-world data is highly dynamic. The statistical distribution of incoming production data can change over time (*Data Drift*), or the actual relationship between the input features and the target variable can evolve (*Concept Drift*).
 - *Impact in MLOps:* The algorithm's predictive accuracy steadily degrades in production, requiring constant monitoring and automated retraining pipelines to stay relevant.
- **2. Scalability and High Latency**
 - *Description:* An algorithm that takes 2 seconds to generate a prediction locally may be completely unacceptable when serving thousands of real-time requests per second in a live application.
 - *Impact in MLOps:* Heavy algorithms (like deep neural networks) struggle to meet strict Service Level Agreements (SLAs) for latency. MLOps teams must heavily optimize, compress, or deploy these algorithms on specialized hardware (GPUs/TPUs).
- **3. Training-Serving Skew**
 - *Description:* This occurs when there is a discrepancy between the data pipeline used to train the algorithm and the data pipeline used to serve the algorithm in production.
 - *Impact in MLOps:* The algorithm receives feature inputs in production that are formatted or scaled slightly differently than its training set, leading to silently incorrect and highly confident bad predictions.
- **4. Reproducibility and Dependency Management**
 - *Description:* Algorithms rely on specific versions of libraries (e.g., scikit-learn, TensorFlow), operating systems, and hardware drivers.

- *Impact in MLOps:* Moving an algorithm to a production server can easily break it if the exact runtime environment is not perfectly replicated. This requires strict containerization (like Docker) to ensure the algorithm behaves consistently.
- **5. Security and Adversarial Threats**
 - *Description:* Exposed algorithms in production APIs are vulnerable to malicious actors.
 - *Impact in MLOps:* Attackers can submit adversarial inputs (data carefully crafted to trick the algorithm) or attempt model inversion attacks to extract the algorithm's internal logic and proprietary data.

3. Conclusion To survive in a production MLOps environment, ML algorithms cannot be treated as static code. They require continuous monitoring frameworks, automated CI/CD pipelines, strict version control, and scalable serving infrastructure to overcome these persistent operational challenges.

2(A) What is the need for version management and reproducibility in developing a machine learning model. Give examples (6 Marks)

1. Introduction In traditional software engineering, tracking code changes is sufficient. However, in machine learning, a model's behavior is determined by the combination of code, hyperparameters, environment, and data. Version management and reproducibility ensure that these complex variables are strictly tracked and controlled.

2. The Need for Version Management Version management in ML extends beyond code (Git) to include Data Version Control (e.g., DVC) and Model Registries.

- **Tracking Iterations:** ML is highly experimental. Teams need to track exactly which combination of data and code produced a specific model version to compare results effectively over time.
- **Safe Rollbacks:** If a newly deployed model begins failing in production (e.g., due to data drift), version management allows the MLOps team to instantly rollback to a previous, stable version of the model.
- **Auditing and Compliance:** Regulated industries (like banking or healthcare) require a strict audit trail. Versioning provides proof of exactly what data a model was trained on and what code was used, ensuring transparency.
- *Example:* A data engineering team accidentally updates a database, introducing null values into a training dataset. Because the ML team uses Data Version Control, they can easily checkout the previous, clean version of the dataset and train the model without delay.

3. The Need for Reproducibility Reproducibility is the ability to duplicate an ML experiment and achieve the exact same results (metrics, weights, predictions).

- **Debugging and Troubleshooting:** If a model produces strange results in production, engineers must be able to recreate the exact training environment and data state locally to investigate the bug.
- **Collaboration:** When multiple data scientists work together, they must be able to run each other's code and get the same results without facing "it works on my machine" issues.

- **Validation of Results:** It ensures that a high-performing model wasn't just a fluke caused by a lucky random seed, but rather a robust, verifiable result.
- *Example:* A data scientist achieves a 98% accuracy on a fraud detection model but fails to set random seeds and log hyperparameters. When they leave the company, the rest of the team cannot reproduce the 98% model to update it with new fraud patterns, rendering the previous work useless.

4. Conclusion Version management and reproducibility are the bedrock of professional MLOps. They transition machine learning from an ad-hoc, individual research task into a reliable, collaborative, and enterprise-ready engineering discipline.

2(B) Debate any two the challenges in choosing evaluation metrics when comparing different models for ML Model. Give examples to justify the answer. (4 Marks)

1. Introduction Evaluation metrics (like Accuracy, Precision, Recall, F1-Score, RMSE) quantify how well a model performs. However, choosing the "best" metric is highly debatable because relying on the wrong metric can lead to deploying a model that appears highly successful on paper but fails disastrously in the real world.

2. Challenge 1: The Accuracy Paradox on Imbalanced Datasets

- **The Debate:** Relying solely on standard "Accuracy" is extremely dangerous when one class of data heavily outnumbers another. Models can achieve high accuracy by simply ignoring the minority class entirely.
- **Justification & Example:** Consider a credit card fraud detection model where 99.9% of transactions are legitimate and 0.1% are fraudulent. A naive, broken model that simply predicts "Not Fraud" for every single transaction will score 99.9% Accuracy. However, this model is completely useless because it fails to catch any fraud. In this scenario, evaluating models based on Accuracy is flawed; one must debate and choose metrics like **Recall** (to catch as many frauds as possible) or the **F1-Score** (to balance false positives and false negatives).

3. Challenge 2: Misalignment with Business Objectives (False Positives vs. False Negatives)

- **The Debate:** Different models make different types of mistakes. Model A might have high Precision (few false alarms) while Model B has high Recall (misses very few actual positive cases). Choosing between them requires debating the real-world cost of a False Positive versus a False Negative, which mathematical formulas cannot decide.
- **Justification & Example:** Consider an ML model used to screen patients for a deadly disease.
 - A **False Positive** means a healthy person is told they might have the disease (causing stress and requiring a follow-up test).
 - A **False Negative** means a sick person is told they are healthy (resulting in a lack of treatment and potential death). Here, the cost of a False Negative is infinitely higher. Therefore, comparing models using a metric that favors Precision (minimizing false

alarms) would be dangerous. The team must choose a metric that optimizes for **Recall** (minimizing missed diagnoses), even if it lowers the overall accuracy or precision.

4. Conclusion Metric selection is not merely a mathematical exercise; it is a critical business decision. The primary challenge lies in ensuring that the chosen mathematical metric accurately reflects the real-world operational goals, class distributions, and cost-benefit trade-offs of the specific ML application.

3(A) XYZ Bank Case Study: Implementing an MLOps Pipeline for Fraud Detection (10 Marks)

1. Scenario Overview XYZ Bank struggles with manual, error-prone deployment and monitoring of its fraud detection model, leading to delayed releases, lack of production visibility, and limited automation. Implementing an automated MLOps pipeline is required to ensure secure, rapid, and observable model deployments.

(i) Various MLOps Tools to Implement the Pipeline

To automate the deployment and monitoring process, the bank can utilize a combination of specialized MLOps tools categorized by their function in the pipeline:

- **Source Code & Data Versioning:** Git (for code), DVC (Data Version Control for datasets).
- **CI/CD Automation:** GitHub Actions, GitLab CI, or Jenkins (to automate testing and deployment workflows).
- **Model Registry & Tracking:** MLflow, Weights & Biases (W&B), or ClearML (to log model versions, hyperparameters, and artifacts).
- **Containerization & Orchestration:** Docker (for packaging) and Kubernetes (for scalable deployment).
- **Model Serving:** Seldon Core, KServe, or FastAPI (to expose the model as a REST/gRPC API).
- **Monitoring & Observability:** Prometheus and Grafana (for system metrics like latency), and Evidently AI or Whylogs (for data/concept drift detection).

(iii) Preferred Deployment Tool Justification and QA/Testing Implementation

Preferred Tool for Deployment: Kubernetes (with Docker & Seldon Core)

- **Justification:** For a financial institution like XYZ Bank, transaction volume can be massive and highly unpredictable (e.g., holiday shopping spikes). **Kubernetes** is the preferred choice because it offers robust **auto-scaling**, ensuring the fraud API remains responsive under heavy load. Combined with **Seldon Core** (an MLOps framework built on Kubernetes), the bank gains enterprise-grade features like out-of-the-box advanced deployment strategies (Canary, Shadow), secure network isolation, and deep integration with monitoring tools.

Elaboration: Implementing the MLOps Pipeline with Quality Assurance (QA) and

Production Testing Implementing the pipeline securely requires integrating QA and testing at every stage—from code commit to post-deployment.

- **Phase 1: Pre-Production Quality Assurance (Continuous Integration - CI)** When a data scientist commits a new fraud model, the CI pipeline automatically triggers tests *before* deployment is allowed.
 - *Unit Testing*: Automated tests ensure the feature engineering scripts correctly process bank transactions (e.g., calculating the distance between successive transactions without crashing on missing data).
 - *Model Validation*: The model is evaluated against a fixed holdout dataset. QA thresholds dictate that the new model must maintain a strict **Recall** score (to ensure it doesn't miss fraud) before it can be packaged.
- **Phase 2: Safe Deployment Testing in Production (Continuous Deployment - CD)**
Deploying immediately to 100% of live traffic is too risky for a bank. The pipeline uses Kubernetes/Seldon to implement safe rollout strategies:
 - *Shadow Deployment (Silent Testing)*: The new model is deployed alongside the old one. Live transactions are sent to *both* models, but only the old model's prediction is used to block transactions. QA teams analyze the new model's hidden predictions to verify it behaves correctly on real-world data.
 - *Canary Release*: Once shadow testing passes, the CD pipeline routes just 5% of real user traffic to the new model. If system metrics (latency, error rates) remain stable, the pipeline gradually increases traffic to 100%.
- **Phase 3: Post-Deployment Monitoring (Continuous Observation)** QA does not end at deployment. Production testing is ongoing.
 - *System Monitoring*: Prometheus tracks the health of the container, ensuring the API responds in milliseconds (critical so legitimate card swipes aren't delayed).
 - *Drift Detection*: Tools like Evidently AI continuously compare live transaction data against the original training data. If fraud patterns change (Concept Drift), automated alerts notify the team that the model's quality is degrading, triggering a retraining cycle.

MLOps QA Pipeline Workflow (Diagram)

Developer Commits New Fraud Model

↓ CI: Unit Tests & Recall Metric Validation (QA Pass/Fail)

↓ Docker Container Built & Registered

↓ CD: Shadow Deployment (Test logic on live data silently)

↓ CD: Canary Release (Rollout to 5% → 100% traffic)

↓ Continuous Monitoring (Alerts on Data Drift or Latency)

Conclusion By leveraging Kubernetes and a structured CI/CD pipeline, XYZ Bank eliminates manual errors, drastically speeds up release cycles, and uses rigorous shadow/canary testing to ensure zero-downtime and high-accuracy fraud detection in production.

4(A) Paraphrase the statement "runtime environments can have a big effect on deployment of ML Deployment and Production" with reference to adaptation of runtime environment from development to production. (6 Marks)

1. Introduction A runtime environment consists of the underlying hardware, operating system, programming language version, and specific software libraries (like TensorFlow or scikit-learn) required to execute a machine learning model.

2. Paraphrasing the Statement The statement essentially means: *"The specific software and hardware configurations used to run a model heavily dictate whether that model will successfully transition from a local testing phase into a stable, live, user-facing system without breaking or slowing down."*

3. Adaptation from Development to Production (The Effects) Adapting a model from a data scientist's environment to a production server introduces significant challenges if the runtime is not carefully managed:

- **Dependency and Version Conflicts:**
 - * *Development:* A data scientist builds a model on their laptop using Python 3.10 and Pandas 2.0.
 - *Production:* The production server runs an older, enterprise-stable Python 3.8 and Pandas 1.3.
 - *The Effect:* Without adaptation, the model will instantly crash in production due to unrecognized functions or missing dependencies.
- **Hardware Discrepancies (CPU vs. GPU):**
 - *Development:* The model might be trained on high-end local GPUs.
 - *Production:* If deployed to a standard cloud CPU instance without optimizing the inference code, the prediction latency could be hundreds of times slower.
- **Scale and Concurrency Limits:**
 - *Development:* The runtime handles one prediction request at a time (batch processing).
 - *Production:* The runtime must adapt to handle thousands of concurrent REST API requests. Standard Python runtimes struggle with this without adaptation tools like Gunicorn or container orchestration.

4. Workflow of Runtime Adaptation (Diagram)

Local Development (Specific OS, Python libraries)

↓ Capture Dependencies (e.g., `requirements.txt`)

↓ Containerization (e.g., Docker packages code + OS + libraries)

↓ Standardized Runtime Created (The "Container Image")

↓ Production Deployment (Container runs exactly the same anywhere)

5. Practical Example A deep learning image classifier is trained on a Windows machine. If the deployment engineer attempts to directly run the raw `.py` files on a production Linux server, pathing issues and missing C++ compiled libraries will cause it to fail. Adapting the environment by encapsulating the exact Windows-equivalent dependencies inside a Docker container ensures the model "believes" it is still running in its original environment, preventing deployment failures.

6. Conclusion The runtime environment is the bridge between ML code and actual business value. Seamless adaptation—primarily achieved through strict dependency tracking and containerization—is mandatory to ensure that models remain stable, secure, and performant when deployed to production.

4(B) Discuss the need of reproducibility and auditability in building a MLOps pipeline. (4 Marks)

1. Introduction Unlike traditional software, machine learning systems are probabilistic and heavily dependent on massive, ever-changing datasets. In this context, building an MLOps pipeline requires strict guarantees that models can be recreated perfectly (**reproducibility**) and that every decision or change can be traced back to its origin (**auditability**).

2. The Need for Reproducibility Reproducibility means ensuring that if an ML experiment is re-run with the same data and code, it produces the exact same model weights and predictions.

- **Effective Debugging:** If a model begins making terrible predictions in production, engineers must be able to roll back and recreate the exact training environment locally to investigate what went wrong. Without reproducibility, the bug cannot be isolated.
- **Team Collaboration:** Data scientists need to share models across teams. Reproducibility eliminates the "it works on my machine" problem, ensuring seamless handoffs from the data science team to the ML engineering deployment team.

3. The Need for Auditability Auditability (or Lineage) is the ability to record the "who, what, when, and how" for every artifact in the ML lifecycle.

- **Regulatory Compliance:** In regulated industries (finance, healthcare), laws like GDPR or AI safety acts require organizations to explain *why* an AI made a specific decision (e.g., denying a loan). Auditability provides a paper trail showing exactly what training data, algorithm version, and parameters were used.
- **Security and Trust:** By maintaining a strict audit trail, organizations can track if malicious data was introduced into a pipeline (data poisoning) or verify that a model was thoroughly tested before it was released to the public.

4. MLOps Traceability Flow (Diagram)

Data Ingestion (Versioned via DVC) —

ML Code & Configs (Versioned via Git) —> Experiment Logger (e.g., MLflow)

Output Model Artifact ————— (Provides Audit Trail & Reproducibility)

5. Practical Example Consider an AI system used in hospital diagnostics. If the system misdiagnoses a patient, the hospital faces legal liability. **Auditability** allows the hospital to pull logs proving the model underwent rigorous, approved testing and identifies the exact dataset used. **Reproducibility** allows the engineering team to feed the patient's data back into that specific historical model version, recreate the error, and fix the underlying flaw in the next version.

6. Conclusion Without reproducibility, an MLOps pipeline is unstable and unmaintainable. Without auditability, it is legally and ethically indefensible. Together, they form the governance backbone necessary to build trustworthy, enterprise-grade machine learning systems.

5(A) With a relevant example determine the attack and vulnerability that can affect an ML Model? (6 Marks)

1. Introduction Machine Learning models deployed in production face unique security threats that traditional software systems do not. Because models learn from data, manipulating that data or exploiting the mathematical nature of the algorithms exposes specific ML vulnerabilities.

2. Key ML Vulnerability: Lack of Robustness to Input Perturbations Unlike traditional code with explicit `if/else` logic, ML models are highly complex statistical boundaries.

- **The Vulnerability:** Models often rely on features or patterns invisible to humans. A slight, deliberate change to the input data can cause the model to make highly confident, yet completely incorrect, predictions.

3. The Attack: Adversarial Evasion

- **Determining the Attack:** In an adversarial evasion attack, a malicious actor intentionally alters the input data *during the inference phase* (when the model is live). The changes are carefully calculated to trick the ML algorithm while remaining completely undetectable to a human reviewer.

4. Practical Example: Autonomous Vehicle Sign Recognition

- **The Scenario:** An autonomous vehicle uses a Convolutional Neural Network (CNN) to read traffic signs.
- **The Attack:** An attacker places small, seemingly random pieces of black and white tape on a "STOP" sign.
- **The Exploitation:** To a human driver, it is obviously still a STOP sign. However, the attacker has exploited the model's *vulnerability to input perturbations*. The specific placement of the tape alters the pixel gradients just enough that the CNN confidently misclassifies the sign as a "Speed Limit 60" sign.
- **The Consequence:** The autonomous vehicle fails to stop, resulting in a potentially fatal crash.

5. Another Example: Data Poisoning Attack

- **Vulnerability:** The model's reliance on continuous learning or user-generated feedback loops without strict data sanitation.
- **The Attack:** Attackers inject malicious data into the training dataset *before or during* the training phase.
- **Example:** In a spam filter, attackers intentionally submit millions of spam emails and manually mark them as "Not Spam". Over time, the model's weights shift, and it slowly learns to ignore actual spam, rendering the security filter useless.

6. Conclusion ML models are highly susceptible to adversarial evasion and data poisoning because of their inherent statistical vulnerabilities. Protecting them requires defensive distillation, adversarial training (showing the model fake examples during training), and strict anomaly detection on incoming inference requests.

5(B) Consider the scenario in Q3. If you were an MLOps engineer, how would you address a situation where a model behaves unexpectedly in production? (4 Marks)

1. Context of Scenario Q3 In Question 3, XYZ Bank has deployed an automated MLOps pipeline for a **fraud detection model**. Unexpected behavior in this context means the model might suddenly be blocking thousands of legitimate transactions (high false positives) or allowing obvious fraud to slip through (high false negatives).

2. Steps to Address Unexpected Behavior As an MLOps engineer for XYZ Bank, I would follow a rapid, structured incident response plan relying on the pipeline's automation and observability:

- **Step 1: Immediate Mitigation (Rollback or Fallback)**
 - *Action:* Stop the bleeding instantly. Since XYZ Bank handles live finances, I would use the CI/CD deployment tool (e.g., Kubernetes/Seldon) to execute a one-click **Rollback** to the previously known stable version of the model.
 - *Fallback:* If the previous version is also failing, I would route traffic to a static rules-based engine (e.g., "Block transactions over \$10,000") until the ML issue is resolved.
- **Step 2: Investigate Logs and Metrics (Observability)**
 - *Action:* I would check the centralized monitoring dashboards (Grafana/Prometheus).
 - *Check System Health:* Is the model receiving corrupted data (e.g., empty fields from the front-end application)? Are there massive latency spikes causing prediction timeouts?
 - *Check Data Integrity:* I would use Python logs to verify the exact JSON payloads the model is receiving to ensure the API format hasn't broken.
- **Step 3: Analyze for Drift (Root Cause)**
 - *Action:* I would consult the drift detection tool (e.g., Evidently AI).
 - *Analysis:* If the system health is fine, the unexpected behavior is likely **Concept Drift**. For example, a new legitimate buying trend (like a massive video game release causing

sudden international transactions) might look like fraud to the current model.

- **Step 4: Reproduce, Retrain, and Redeploy**

- *Action:* Once the root cause is found (e.g., shifted data patterns), I pull the recent anomaly data using Data Version Control (DVC). The data science team retrains the model on these new patterns. The updated model is pushed through the CI/CD pipeline, passing automated tests and a Canary release, before fully replacing the